Scikit

David Arroyo Menéndez

April 28, 2026

Get Scikit Examples

You can download the source from:

```
$ git clone https://github.com/davidam/damescikit
```

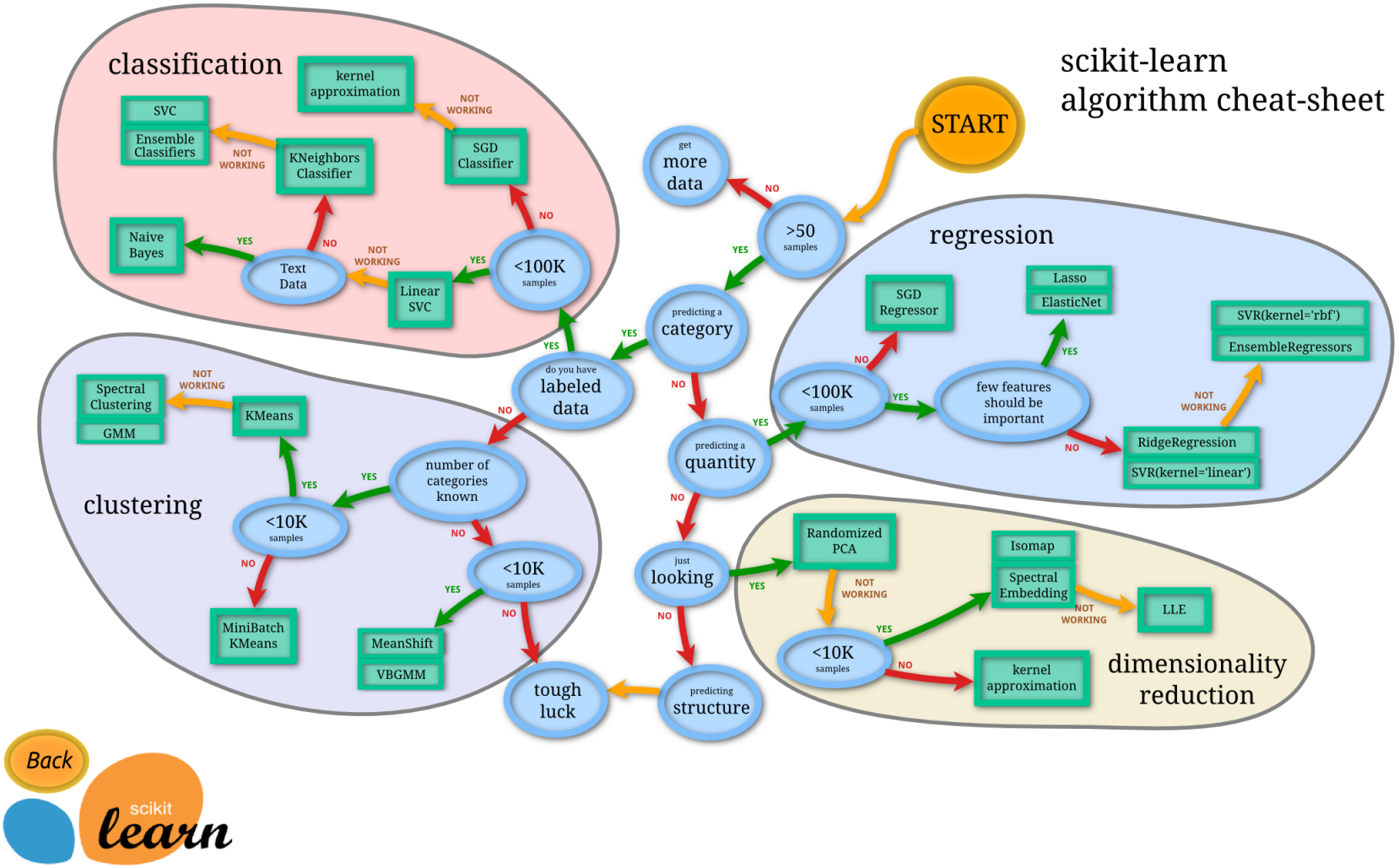
Or you can download from pypi:

```
$ pip3 install damescikit
```

Summary of Examples

1. Classification
2. Clustering
3. Regression
4. Dimensionality Reduction

Scikit Graph



Datasets

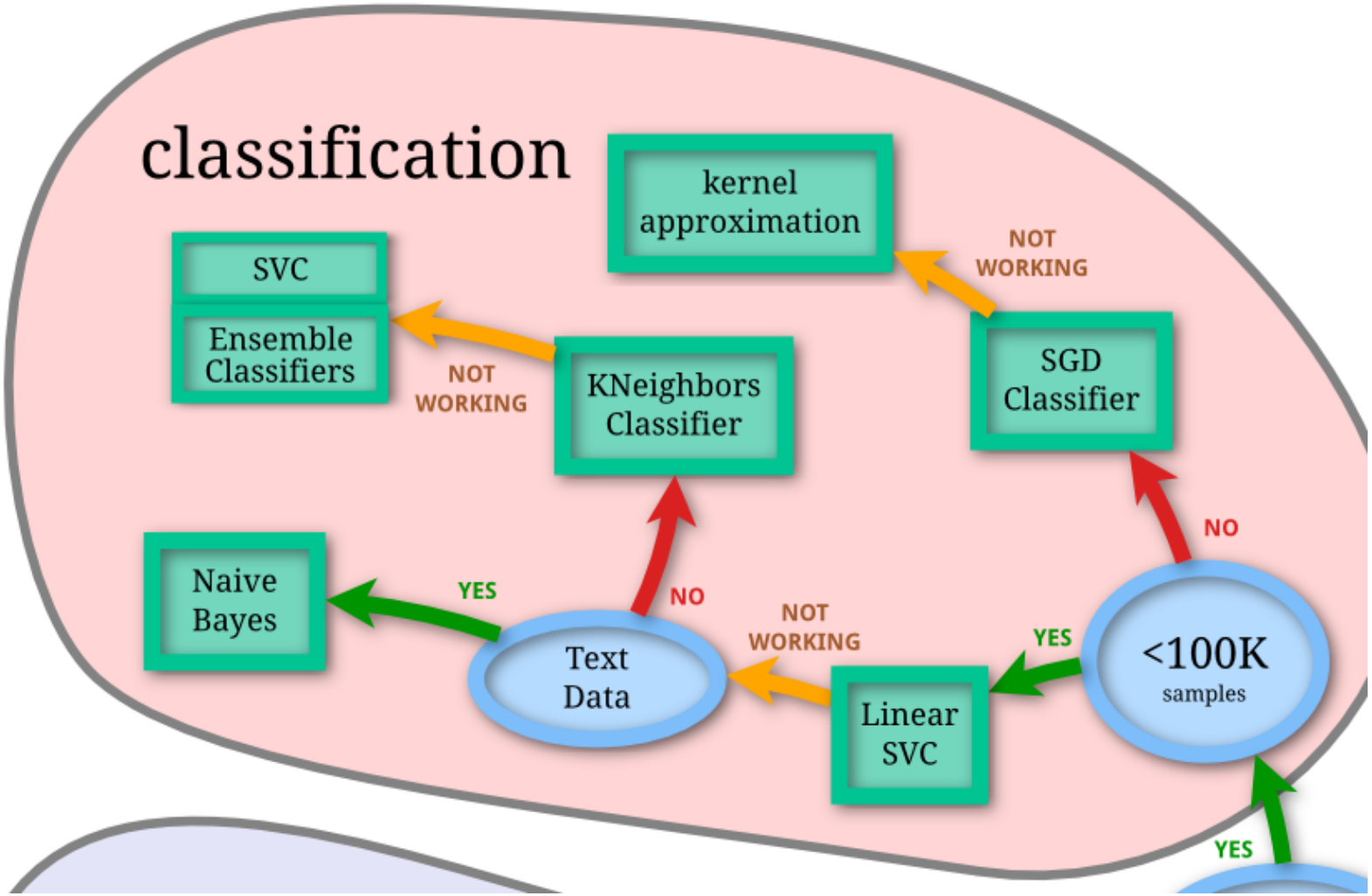
Any machine learning exercise requires datasets. You can load datasets with scikit. If you are creating a new dataset you can use $\text{get}_{\text{datahome}}$:

```
from sklearn.datasets import get_data_home
        data_path = os.path.join(get_data_home(), "reuters")
```

You can create automated datasets, or use sklearn datasets.

```
$ python3 plot_random_dataset.py
$ python3 rcv1.py
$ python3 lfw.py
```

Classification



Classification: SGD

We want predict a category and we've labeled data with <100k samples

```
$ python3 plot_sgd_iris.py
$ python3 sgd.py
```

Classification: Kernel Approximation

We want predict a category and we've labeled data with <100k samples and SGD is not working

```
$ python3 kernel-approximation.py
```

Classification: Naive Bayes

What we want to know; the Posterior probability of Class j given a predictor x

The Likelihood; the probability of the predictor given a Class j. Its computed from the training-set.

The Prior probability of Class j; what we know about the class distribution before we consider x.

$$P(Class_j \mid x) = \frac{P(x \mid Class_j) \times P(Class_j)}{P(x)}$$

The Evidence. In practice, there's interest only in the numerator (denominator is effectively constant)

Applying the independence assumption

$$P(x \mid Class_j) = P(x_1 \mid Class_j) \times P(x_2 \mid Class_j) \times \ldots \times P(x_k \mid Class_j)$$

Substituting the independence assumption, we derive the Posterior probability of Class j given a new instance x' as...

$$P(Class_j \mid x') = P(x'_1 \mid Class_j) \times P(x'_2 \mid Class_j) \times \ldots \times P(x'_k \mid Class_j) \times P(Class_j)$$

Classification: Naive Bayes

	Predictors				Response
	Outlook	**Temperature**	**Humidity**	**Wind**	**Class** **Play=Yes** **Play=No**
Day1	Sunny	Hot	High	Weak	No
Day2	Sunny	Hot	High	Strong	No
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	No
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	No
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	No

Classification: Naive Bayes

P(Outlook=o|Class $_{Play=Yes\|No}$)	Frequency		Probability in Class	
Outlook =	Play=Yes	Play=No	Play=Yes	Play=No
Sunny	2	3	2/9	3/5
Overcast	4	0	4/9	0/5
Rain	3	2	3/9	2/5
	total= 9	total=5		

P(Temperature=t|Class $_{Play=Yes\|No}$)	Frequency		Probability in Class	
Temperature =	Play=Yes	Play=No	Play=Yes	Play=No
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
	total= 9	total=5		

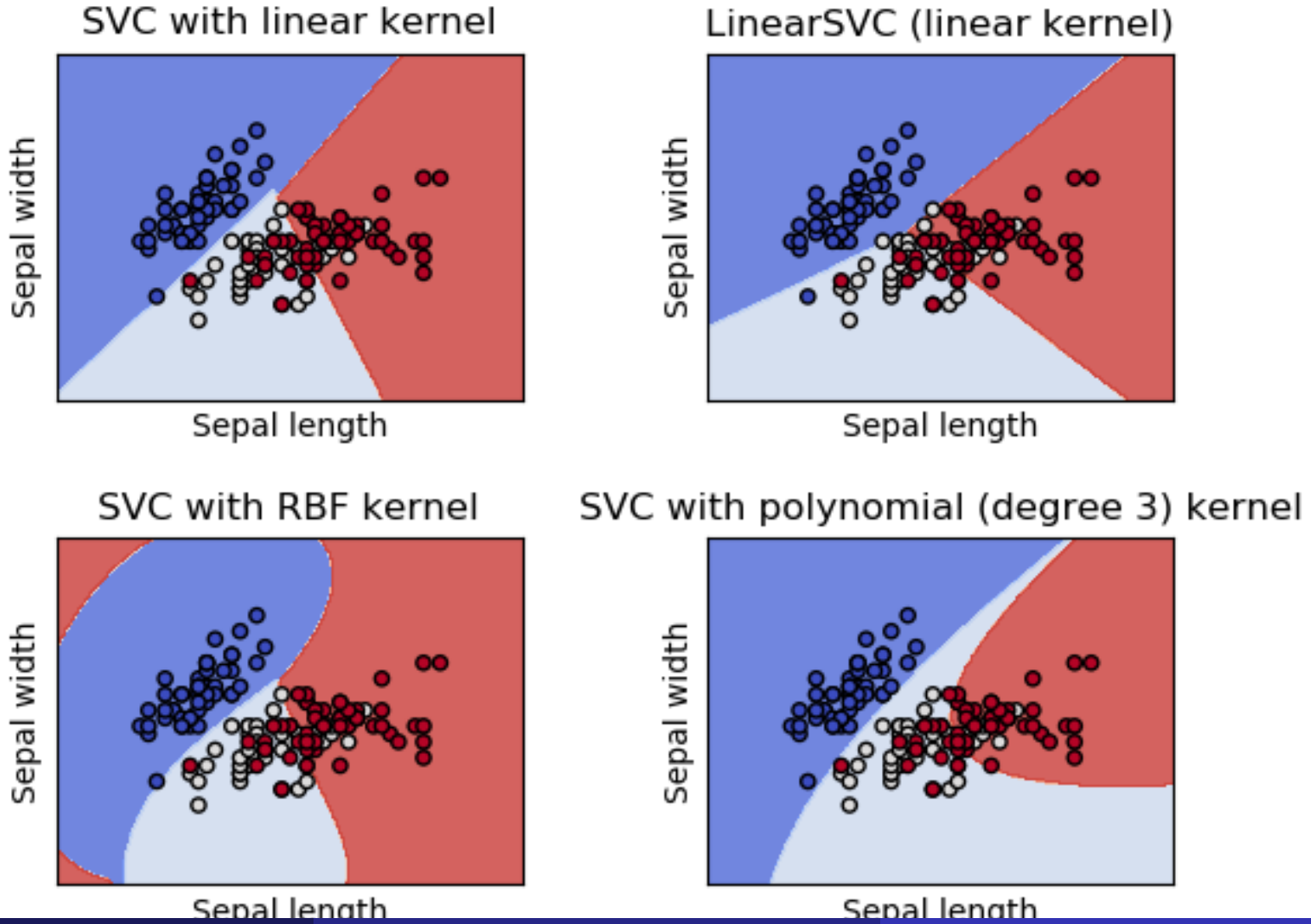
P(Humidity=h|Class $_{Play=Yes\|No}$)	Frequency		Probability in Class	
Humidity =	Play=Yes	Play=No	Play=Yes	Play=No
High	3	4	3/9	4/5
Normal	6	1	6/9	1/5
	total= 9	total=5		

Classification: Naive Bayes Examples

```
$ python3 gaussiannb.py
$ python3 bernoullinb.py
$ python3 multinomialnb.py
$ python3 nltk/sexmachine.py
```

Classification: SVC

We want predict a category and we've labeled data with <100k samples



1.4.6. Kernel functions

The *kernel function* can be any of the following:

- linear: $\langle x, x' \rangle$.
- polynomial: $(\gamma \langle x, x' \rangle + r)^d$. d is specified by keyword `degree`, r by `coef0`.
- rbf: $\exp(-\gamma \|x - x'\|^2)$. γ is specified by keyword `gamma`, must be greater than 0.
- sigmoid ($\tanh(\gamma \langle x, x' \rangle + r)$), where r is specified by `coef0`.

Different kernels are specified by keyword kernel at initialization:

```
>>> linear_svc = svm.SVC(kernel='linear')
>>> linear_svc.kernel
'linear'
>>> rbf_svc = svm.SVC(kernel='rbf')
>>> rbf_svc.kernel
'rbf'
```

Classification. SVC

We want predict a category and we've labeled data with <100k samples

```
$ python3 svc-example.py
```

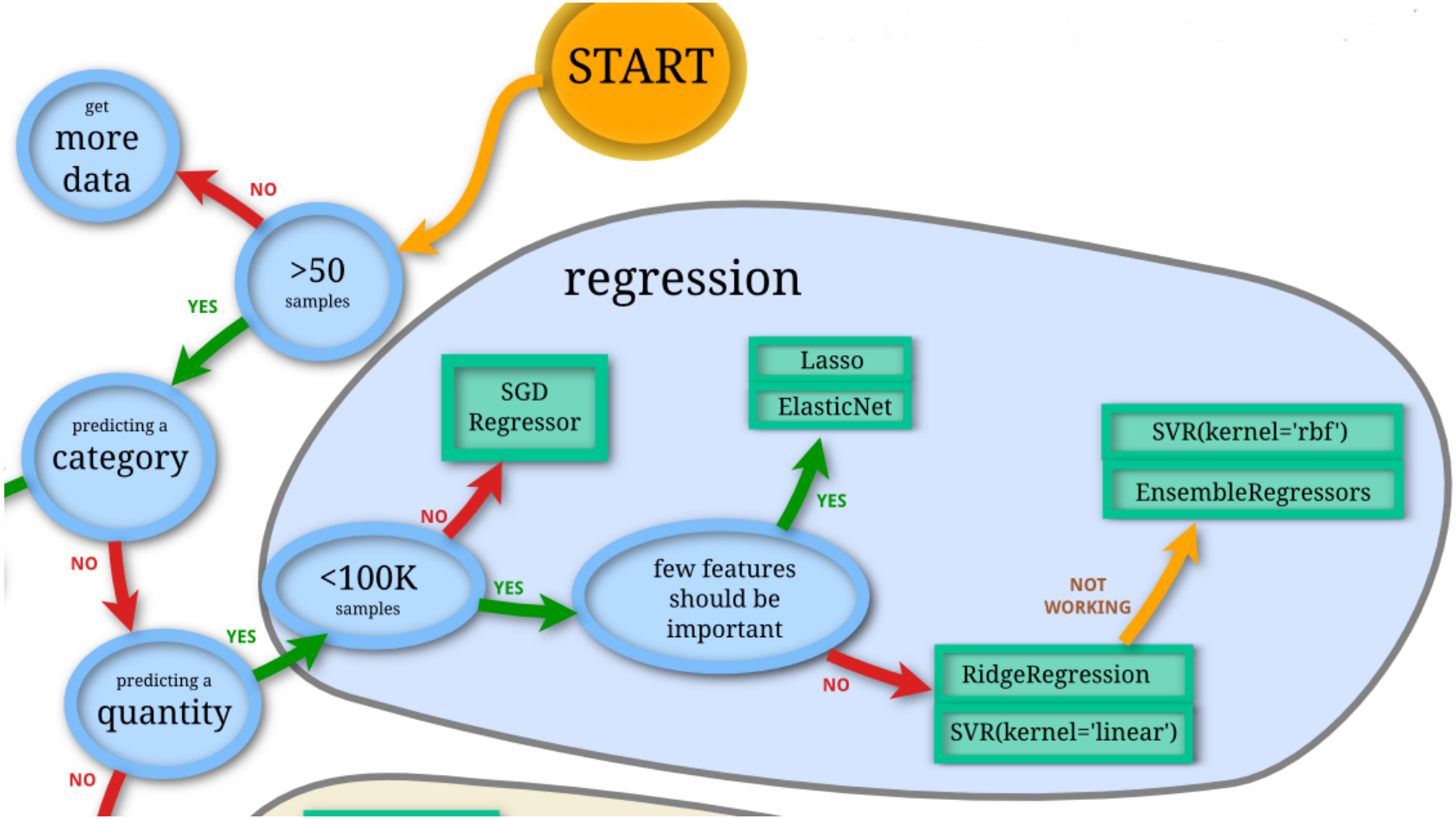
Classification: Trees

ID3: The algorithm creates a multiway tree, finding for each node (i.e. in a greedy manner) the categorical feature that will yield the largest information gain for categorical targets. Trees are grown to their maximum size and then a pruning step is usually applied to improve the ability of the tree to generalise to unseen data.
C4.5: is the successor to ID3 and removed the restriction that features must be categorical by dynamically defining a discrete attribute (based on numerical variables) that partitions the continuous attribute value into a discrete set of intervals.
CART : is very similar to C4.5, but it differs in that it supports numerical target variables (regression) and does not compute rule sets.

Classification: Trees (II)

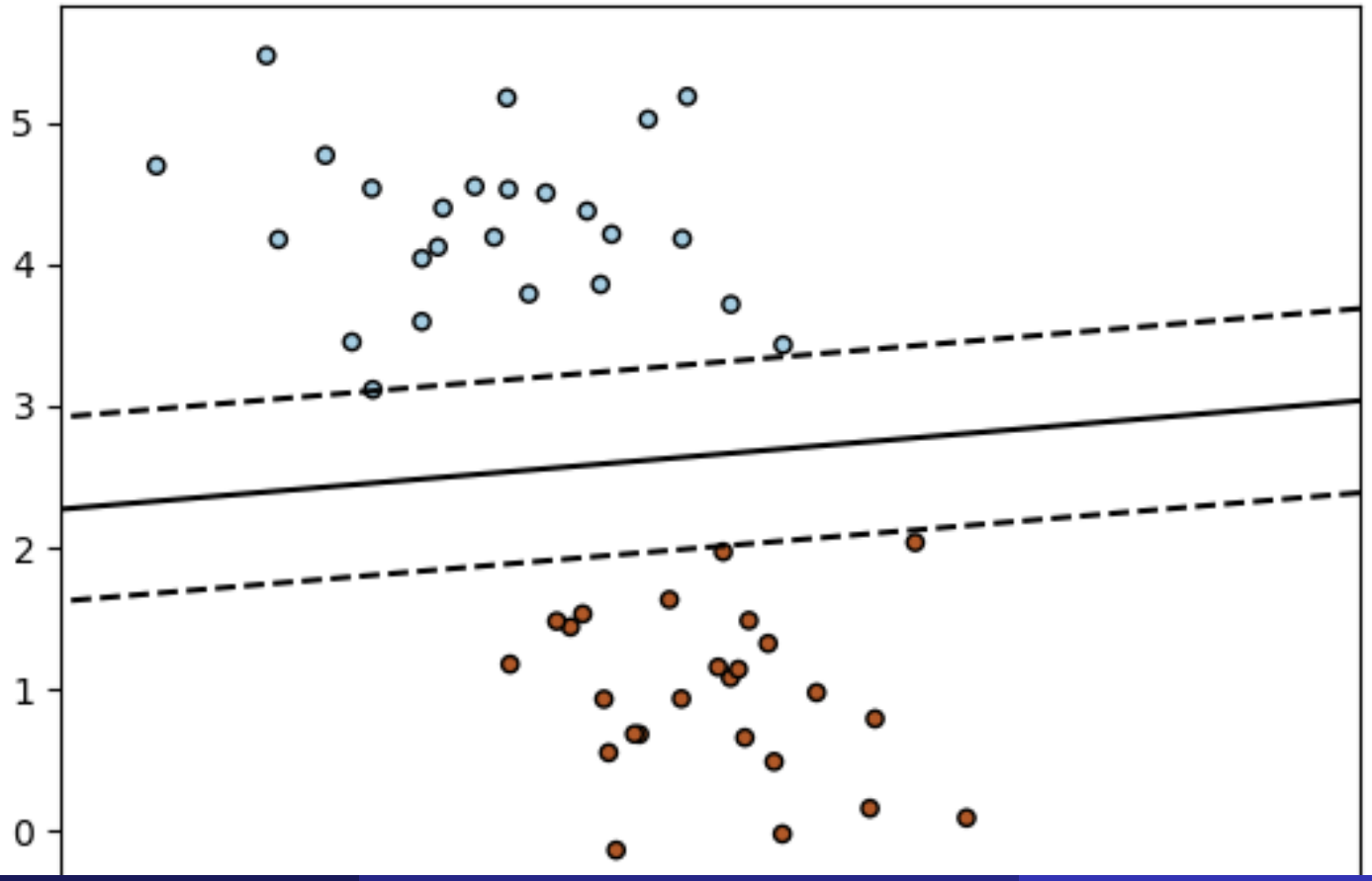
```
$ python3 plot_iris1.py
$ python3 plot_unveil_tree_structure.py
$ python3 plot_classifier_comparison.py
$ python3 plot_ensemble_oob.py
$ python3 datasets/decisiontree.py
```

Regression


START
get more data
NO
>50 samples
YES
predicting a category
NO
predicting a quantity
YES
NO
regression
SGD Regressor
NO
<100K samples
YES
few features should be important
YES
Lasso
ElasticNet
NO
RidgeRegression
SVR(kernel='linear')
NOT WORKING
SVR(kernel='rbf')
EnsembleRegressors

SGD Regressor

The class SGDClassifier implements a plain stochastic gradient descent learning routine which supports different loss functions and penalties for classification.



SGD Regressor (II)

As other classifiers, SGD has to be fitted with two arrays: an array X of size [$n_{samples}$, $n_{features}$] holding the training samples, and an array Y of size [$n_{samples}$] holding the target values (class labels) for the training samples:

```
$ python3 plot_sgd_iris.py
$ python3 sgd.py
```

Lasso

Linear Model trained with L1 prior as regularizer (aka the Lasso) The optimization objective for Lasso is: (1 / (2 * $n_{samples}$)) * $||y - Xw||^2_2$ + alpha * $||w||_1$ Technically the Lasso model is optimizing the same objective function as the Elastic Net with $l1_{ratio}$=1.0 (no L2 penalty).

```
$ python3 plot_lasso_and_elasticnet.py
$ python3 plot_multi_task_lasso_support.py
```

Kernel Ridge

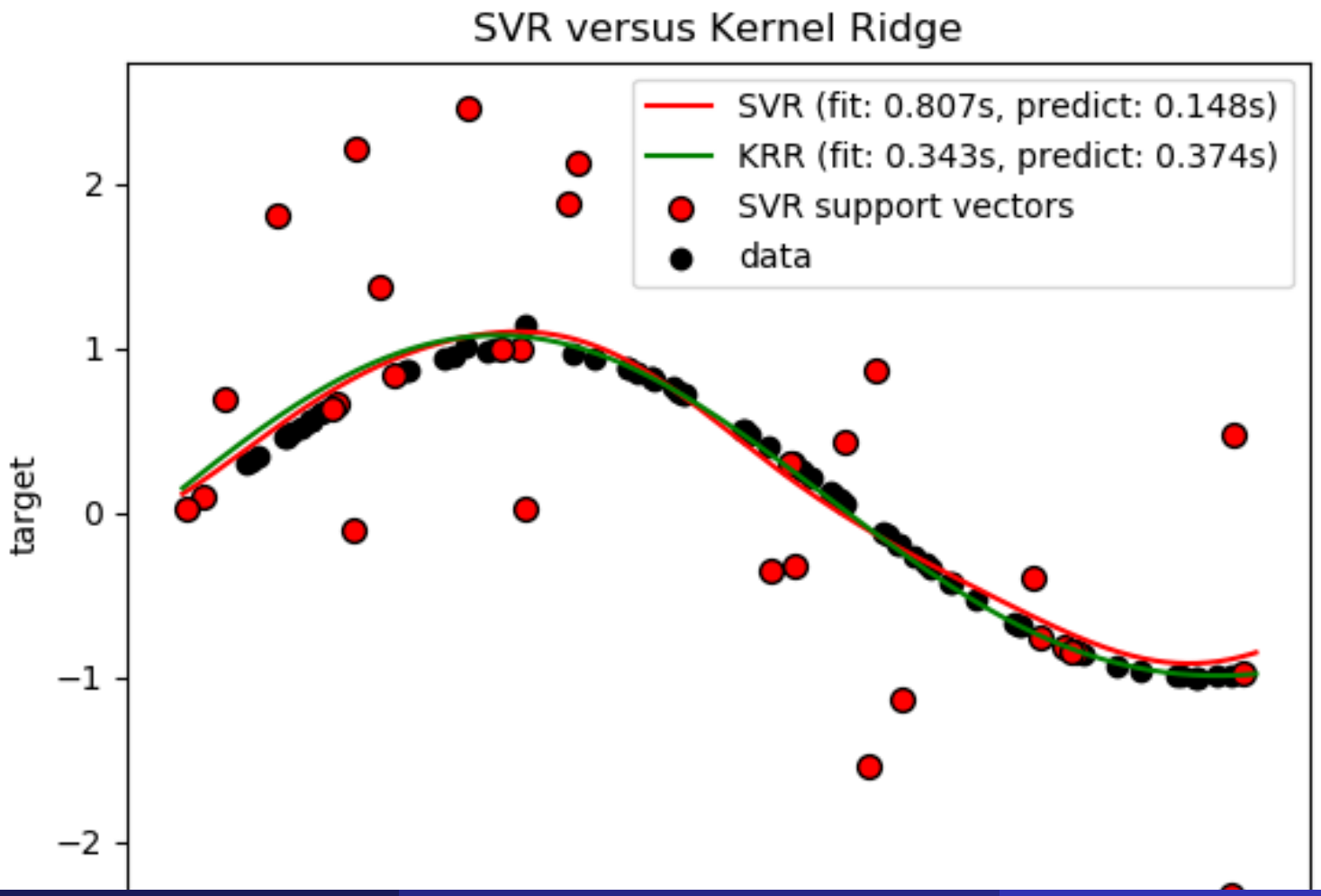
Kernel ridge regression (KRR) [M2012] combines Ridge Regression (linear least squares with l2-norm regularization) with the kernel trick. It thus learns a linear function in the space induced by the respective kernel and the data. For non-linear kernels, this corresponds to a non-linear function in the original space.

```
$ python3 plot_kernel_ridge_regression.py
```

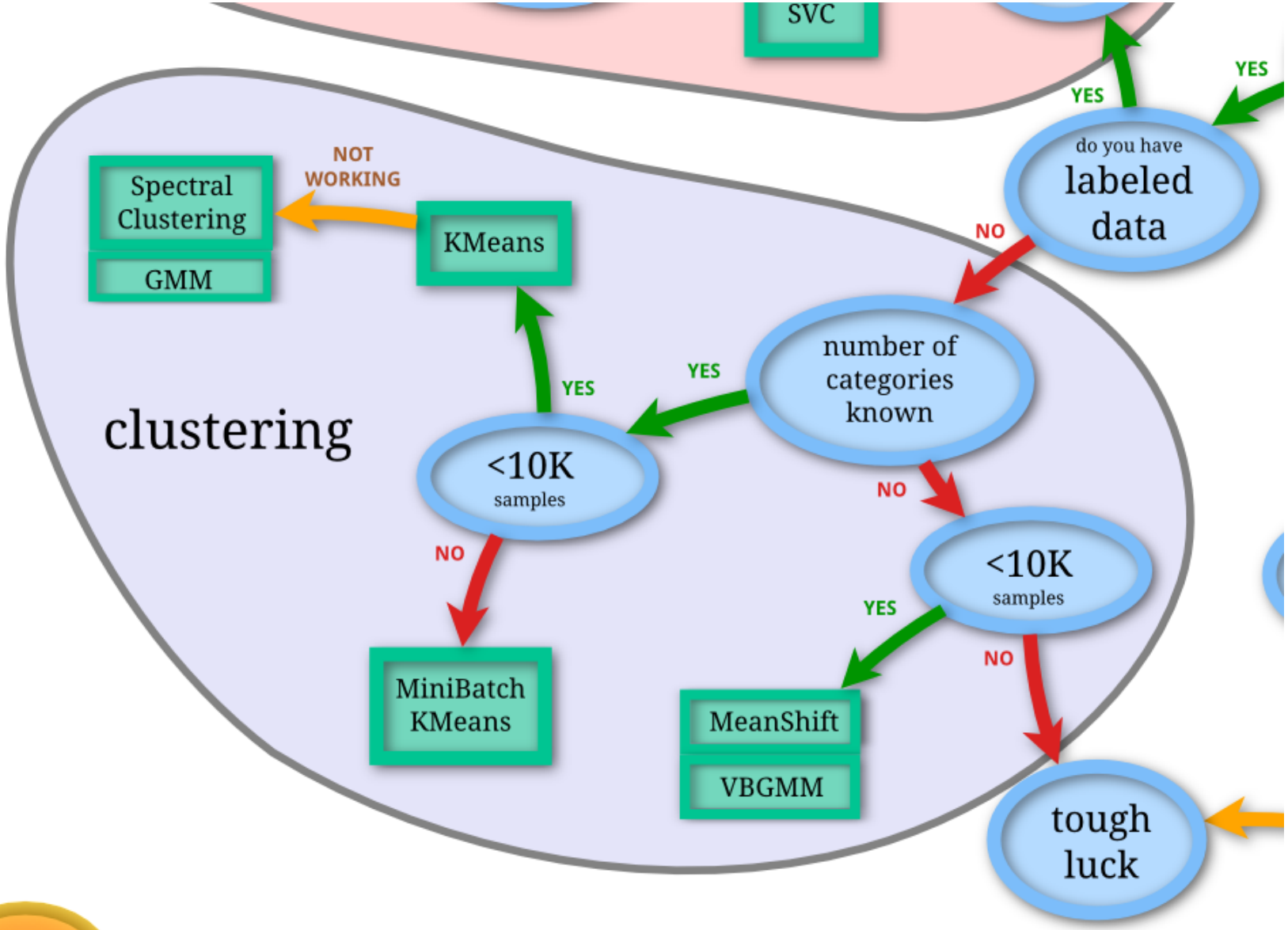
Support Vector Regression

The free parameters in the model are C and epsilon.

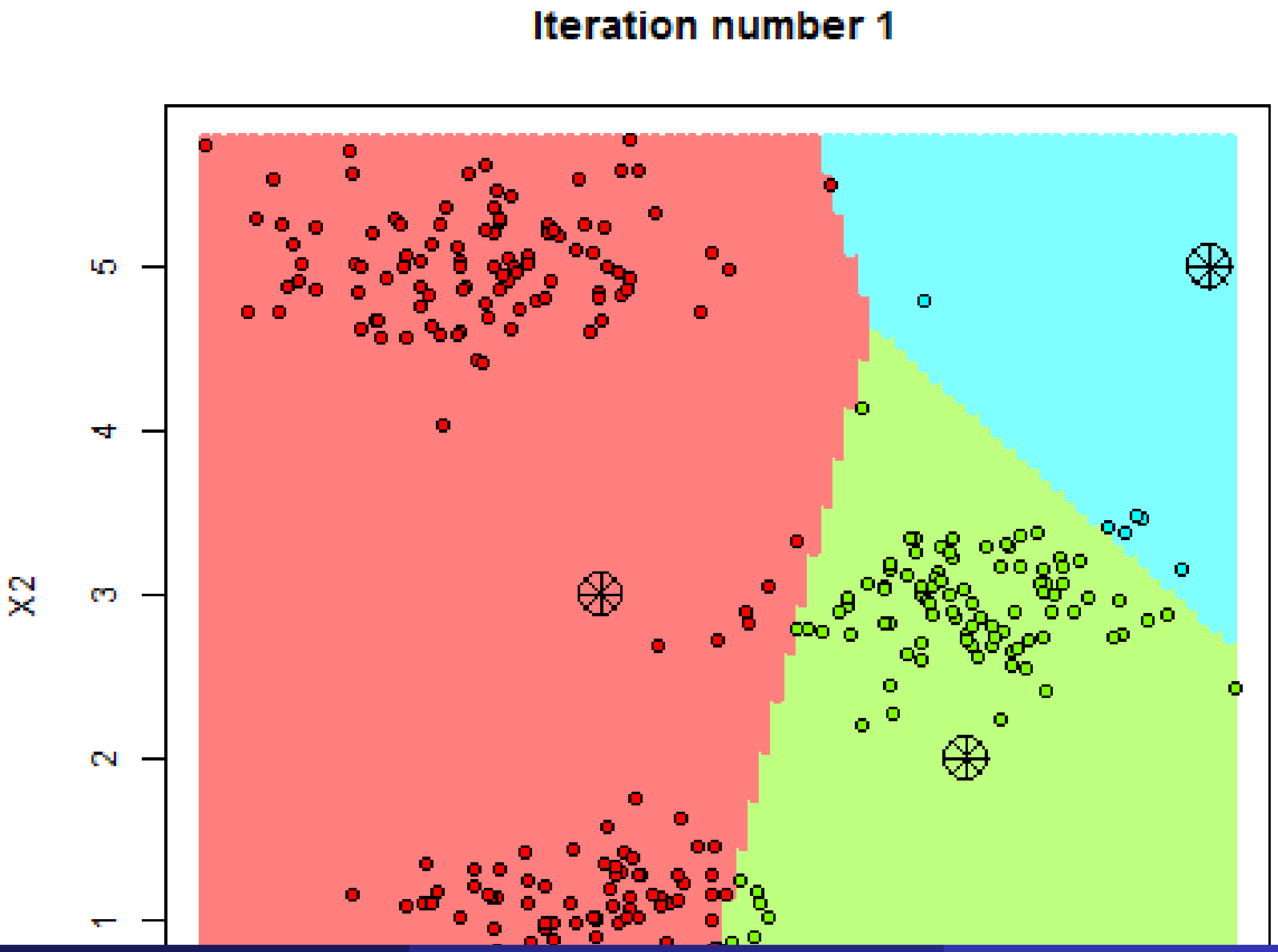
```
$ python3 svr-example.py
```



Clustering


SVC
YES
YES
do you have
labeled
data
NO
number of
categories
known
YES
YES
<10K
samples
NO
<10K
samples
YES
NO
MeanShift
VBGMM
tough
luck
NO
MiniBatch
KMeans
YES
KMeans
NOT
WORKING
Spectral
Clustering
GMM
clustering

Kmeans



Kmeans

Subdivide the space making regions from reference points called centroides

```
$ python3 plot_kmeans_assumptions.py
$ python3 plot_cluster_iris.py
```

GMM

A typical finite-dimensional mixture model is a hierarchical model consisting of the following components:
N random variables that are observed, each distributed according to a mixture of K components, with the components belonging to the same parametric family of distributions (e.g., all normal, all Zipfian, etc.) but with different parameters N random latent variables specifying the identity of the mixture component of each observation, each distributed according to a K-dimensional categorical distribution A set of K mixture weights, which are probabilities that sum to 1. A set of K parameters, each specifying the parameter of the corresponding mixture component. In many cases, each "parameter" is actually a set of parameters. For example, if the mixture components are Gaussian distributions, there will be a mean and variance for each component. If the mixture components are categorical distributions (e.g., when each observation is a token from a finite alphabet of size V), there will be a vector of V probabilities summing to 1.

```
$ python3 plot_gmm_covariances.py
```

Spectral Clustering

Make use of the spectrum (eigenvalues) of the similarity matrix of the data to perform dimensionality reduction before clustering in fewer dimensions

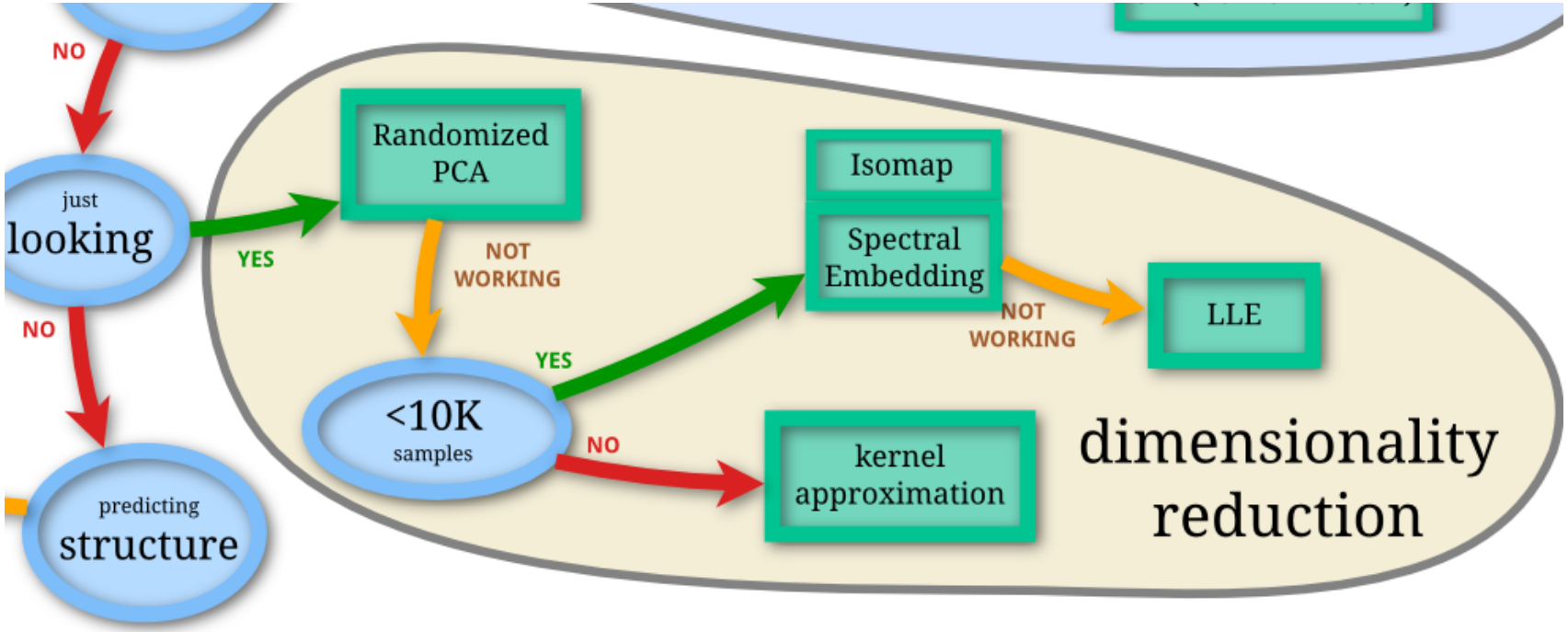
```
$ python3 plot_cluster_comparison.py
```

Mean Shift

Mean shift is a non-parametric feature-space analysis technique for locating the maxima of a density function, a so-called mode-seeking algorithm.

```
$ python3 plot_mean_shift.py
```

Dimensionality Reduction



The general task of pattern analysis is to find and study general types of relations (for example clusters, rankings, principal components, correlations, classifications) in datasets

Randomized PCA

PCA is mostly used as a tool in exploratory data analysis and for making predictive models. It's often used to visualize genetic distance and relatedness between populations. PCA can be done by eigenvalue decomposition of a data covariance (or correlation) matrix or singular value decomposition of a data matrix, usually after mean centering

```
$ python3 pca-example.py
$ python3 pca.py
$ python3 scikit-plot-pca.py
$ python3 plot_pca_iris.py
$ python3 incremental-pca.py
```

Kernel Approximation

The general task of pattern analysis is to find and study general types of relations (for example clusters, rankings, principal components, correlations, classifications) in datasets.
For many algorithms that solve these tasks, the data in raw representation have to be explicitly transformed into feature vector representations via a user-specified feature map: in contrast, kernel methods require only a user-specified kernel, i.e., a similarity function over pairs of data points in raw representation.

```
$ python3 kernel-approximation.py
```

LLE and Spectral Embedding

```
$ python3 plot_lle_digits.py
$ python3 plot_spectral_grid.py
```

Spectral embedding for non-linear dimensionality reduction.
Forms an affinity matrix given by the specified function and applies spectral decomposition to the corresponding graph laplacian. The resulting transformation is given by the value of the eigenvectors for each data point.

Isomap

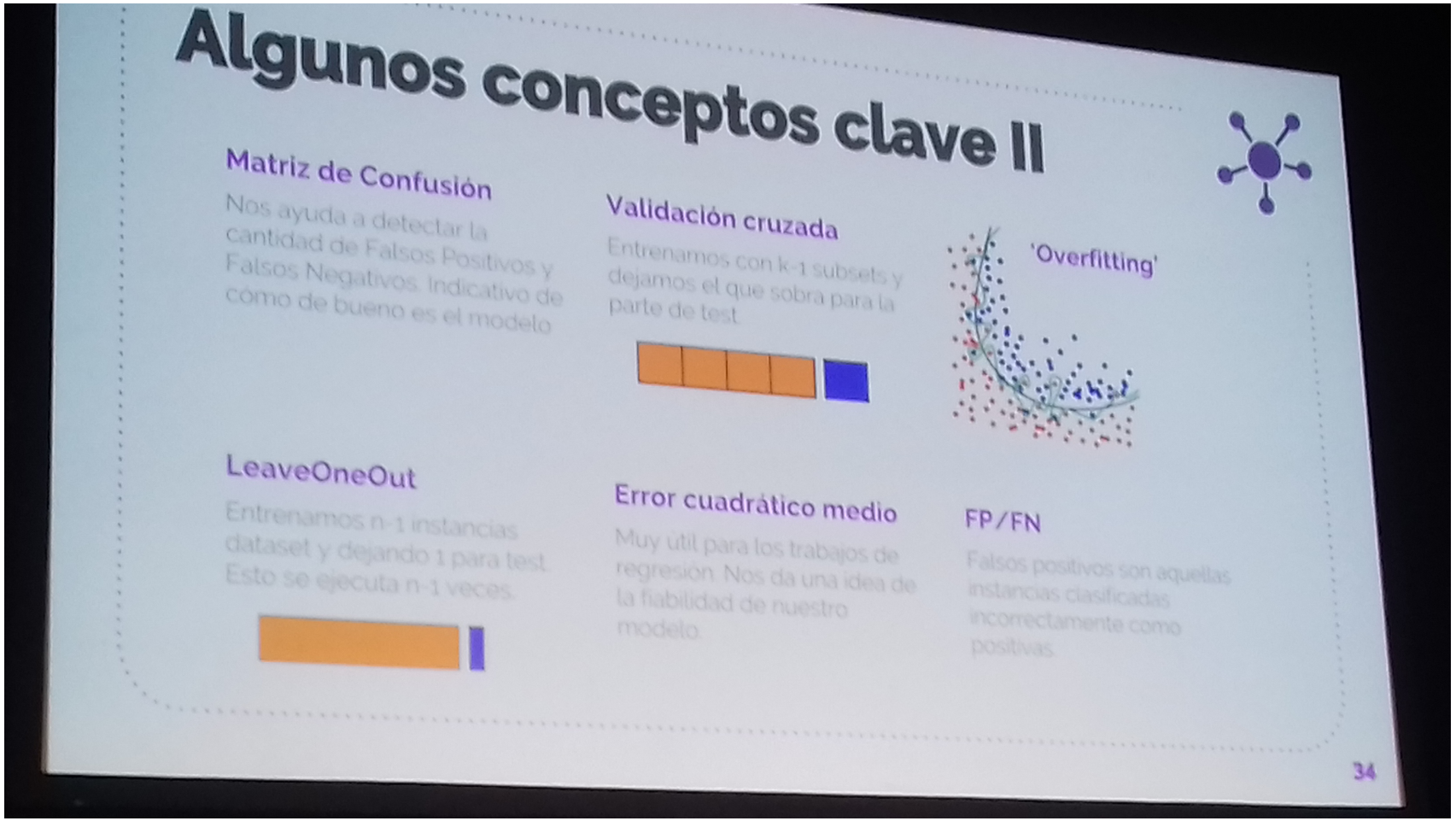
Isomap Embedding Non-linear dimensionality reduction through Isometric Mapping

```
$ python3 plot_compare_methods.py
```

Scikit Image

```
$ python3 plot_marching_cubes.py
```

Conceptos Clave



Confusion Matrix

Each row of the matrix represents the instances in a predicted class while each column represents the instances in an actual class.

```
$ python3 plot_confusion_matrix.py
```

In a confusion matrix you must understand to make an accuracy between the predicted class and the real class.

```
$ python3 accuracy.py
```

This exercise is useful to understand the details to build a confusion matrix

```
$ python3 confusion-matrix.py
```

Leave One Out

```
$ python3 leaveoneout.py
$ python3 leavepout.py
```

Validación Cruzada

```
$ python3 crossvalidation.py
$ python3 repeatedkfold.py
```

Overfitting

Avoiding troubles with additional data

```
$ python3 nooverfitting.py
```

Outliers

Son valores atípicos que se salen del doble de la desviación típica, por ejemplo.

```
$ python3 numpy/reject-outliers.py
$ python3 scikit/plot_outlier_detection_housing.py
```

Pipelines

Es posible hacer pipes dese un algoritmo a otro usando esta librería

```
$ python3 sckikit/pipeline.py
```

Comparing Classifiers

```
$ python3 plot_classifier_commparison.py
```

Dados puntos azules y rojos aprende el espacio donde se sitúan estos.

```
$ python3 plot_compare_methods.py
```

En reducción de dimensiones, podemos imaginar una imagen tridimensional de bolas aplanarla a una imagen bidimensional.

```
$ python3 plot_compare_calibration.py
$ python3 plot_calibration_curve.py
```

Otra manera interesante de comparar reducción de dimensiones es con grid search y pipeline

```
$ python3 plot_compare_reduction.py
```

Otra cuestión son los clasificadores bien calibrados (probabilísticos).
Para aprender y avanzar: `http://benalexkeen.com/scoring-classifier-models-using-scikit-learn/`

Model selection and evaluation (I)

Cross-validation: evaluating estimator performance

- Computing cross-validated metrics: scores (Ej: ['fit_{time}', '$\text{score}_{\text{time}}$', '$\text{test}_{\text{precision macro}}$', '$\text{test}_{\text{recall macro}}$'])
- Cross validation iterators (K-fold, Repeated K-Fold, Leave One Out (LOO), Leave P Out (LPO), ...)

Tuning the hyper-parameters of an estimator

- Exhaustive Grid Search
- Randomized Parameter Optimization
- Tips for parameter search

Model selection and evaluation (II)

Model evaluation: quantifying the quality of predictions

- The scoring parameter: defining model evaluation rules
- Classification metrics
- Multilabel ranking metrics
- Regression metrics

Model persistence

Example: pickle, joblib, ...

Building Machine Learning from scratch

Uno puede construir sus propios algoritmos de machine learning, leyendo desde tutoriales, o artículos científicos

```
$ python3 text-classification.py
$ python3 layerneuralnetwork.py
```